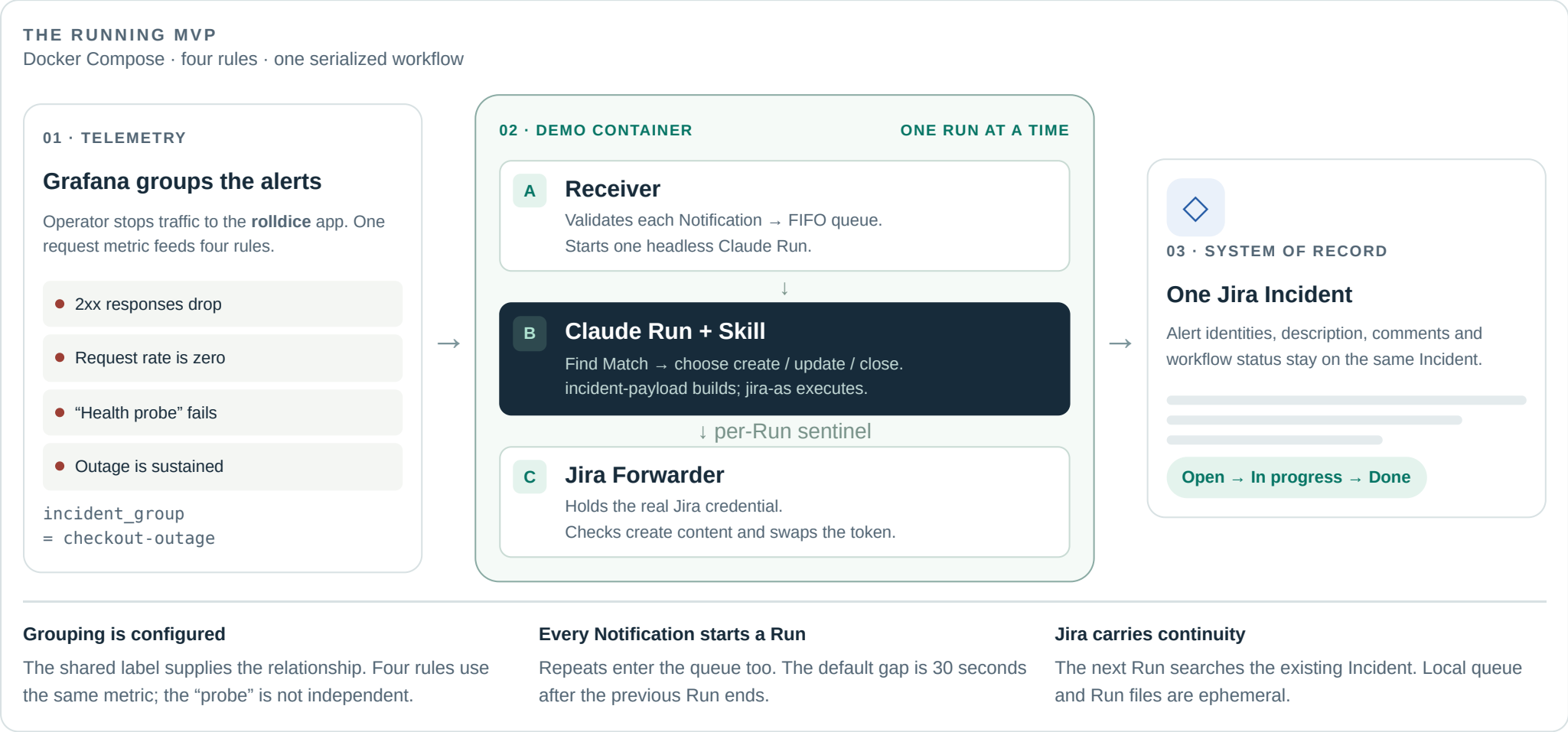


Many alerts. One incident

How the MVP turns a configured group of Grafana alerts into one evolving Jira Incident, with controlled AI execution and a clear path to stronger evidence.

01

Follow the alert to the Incident



Watch one Incident evolve

The open Match uses project, Incident type, group and session. Each new Alert adds a fingerprint; later Notifications add comments to the same Incident.

CLICK THROUGH THE LIFECYCLE

Illustrative state · recorded demo timings, not a service-level promise

One incident through the cycle

Stage	Action on the same Incident	Resulting state	Fingerprints
Create ~75 s	No open Match. Create one Incident with a description, group + session labels, two fingerprints and an opening comment.	Open	2
Related alert ~135 s	The “health probe” joins. Add its fingerprint and a New / Repeat comment. Move Open to Work in progress.	In progress	3
Sustained ~195 s	The sustained-outage Alert joins. Add its fingerprint and another update to the same Incident.	In progress	4
Repeat ~375 s · optional	No new fingerprints. Record a repeat comment on the same Incident. An unchanged repeat still consumes a Run.	In progress	4
Recover After traffic restarts	All Alerts in the current Notification are resolved. Comment and complete the Incident; read back status and resolution.	Completed	4

Completion applies to managed workflow states; other human-owned statuses are left to the human. Fingerprints record Alerts ever seen, rather than durable current firing state. Recovery time includes Grafana notification, queue and Run time.

Why one Incident? Shared labels identify the group; the session isolates a take; fingerprint labels identify Alerts; serial processing reduces create races. The MVP follows this explicit identity contract.

Separate judgment from mechanics

The model selects actions within the prescribed workflow. Deterministic code handles the fragile payload work. The operator and responder retain service ownership.

WHO DECIDES AND WHO ENFORCES

Current MVP boundary



DETERMINISTIC CODE

Make mechanics reliable

Grafana groups by the explicit label; the Receiver serializes Runs.

incident-payload builds JSON, labels and comment text.

The Forwarder admits one create attempt with the expected summary, description and label set.



CLAUDE RUN

Select the next action

Read the Skill and search for the prescribed open Match.

Choose create, update, close or skip; execute generated commands.

When enabled, choose read-only telemetry queries and state observations, interpretation and unknowns.



OPERATOR AND RESPONDER

Own the service

Configure grouping, Jira workflow, credentials and demo budget.

Inject the fault and restore traffic.

Inspect evidence, assess suggested causes and decide remediation.

Contained execution

Non-root container, read-only root, dropped capabilities, resource limits and a narrow Claude tool allowlist. Run scratch and queue are ephemeral.

Credential mediation has a specific scope

The Run is passed a Jira sentinel. It directly holds the model credential and optional Grafana token. Jira account permissions still bound broader calls; this Forwarder is not a complete authorization policy.

One-at-a-time execution and create checks support the demonstrated behavior. They do not establish durable exactly-once processing or a production security boundary.

Understand what the demo establishes

Recorded lifecycle success is the foundation. Current interface fixes, optional investigation and the future operating model each need evidence appropriate to their scope.

READ THE EVIDENCE IN LAYERS

Repository snapshot · 2 Oct 2026

● Recorded live proof

01

The incident lifecycle works

- Real Jira + real Claude replay and live traffic-stop runs reached VERIFIED.
- Three model rehearsals kept one Incident per cycle.
- Content quality differed; lifecycle success alone missed a placeholder description.

● Acceptance still needed

02

Current fixes and investigation

- Payload helper, create guard and stronger content checks were offline tested after those rehearsals.
- That findings record does not establish a post-fix live rerun.
- Optional metrics / logs / traces need their own real-model and real-Jira acceptance.

● Outside this MVP

03

Broader operating model

- Evidence-based matching across a distributed cascade.
- Durable recovery, reviewed memory and Confluence learning.
- Kubernetes deployment, stronger service mediation and production scale.

WHERE THE WAIT COMES FROM · DEMONSTRATION TIMING

~75 seconds

stop → first grouped Notification

+

Queue + settling

30-second default gap between Runs

+

Run execution

often 20–30 s; model and work vary

Timing components vary and the first Run may have no queue or settling delay. Optional investigation adds work. No measured responder time savings or autonomous root-cause discovery is established.

What to validate next

LEADERSHIP QUESTION	RECOMMENDED NEXT EVIDENCE
Can a cheaper model be trusted?	Replay identical fixtures after the fixes. Compare final Incident content, failed commands, elapsed time and cost.
Does investigation help responders?	Validate faithful evidence on the same real Incident, test unavailable telemetry, and measure added latency, queue delay and cost.
What blocks wider adoption?	Explicitly accept durable recovery, throughput and access boundaries. The in-memory queue and single worker trade simplicity for resilience and scale.
01 · MVP behavior and verification contract	02 · Current runbook and investigation limits
03 · Rehearsal evidence and post-fix gaps	04 · Actual queue and Run scheduling
05 · Run credentials and Jira permission caveats	06 · Container runtime and network setup
07 · Skill matching and workflow behavior	08 · Chapter two scope and status

Grounded in repository revision f910a5b, reviewed 2 October 2026. This explainer simulates no live service and makes no external API calls. Timings describe the documented demo, not a benchmark. Recommended evidence is editorial guidance.